



VU Enrolment & Timetable Allocation System

Written Report — Assessment 2

Members:

Jenish Gurung

Hoang Gia Phat Nguyen

Table of Contents

Project Summary.....	3
1. The VU Enrolment System, In Our Own Words.....	3
2. Enrolment Actions as Operating-System-Managed Activities.....	3
2a. Operating System Evolution and Social Impact.....	3
3. System Constraints and Their Effect on Enrolment Outcomes.....	4
4. Trade-offs.....	4
Section A — Multi-Constraint Reasoning.....	4
Section B — Strategy Selection Under Conflict.....	5
Section C — Simulation with Partial Failure.....	5
Section D — Reflection.....	6
Team Coordination.....	6
Individual Contribution.....	6
Personalised Enrolment Request That Cannot Be Fulfilled.....	7

Project Summary

This report presents the design, simulation, and evaluation of a scheduler for the VU online enrolment and timetable allocation system, modelled as an operating system resource-allocation problem. Enrolment requests are treated as processes competing for two constrained shared resources — CPU time and memory — under an additional, non-negotiable admission gate: class seat capacity.

Our approach combines Round Robin CPU scheduling (quantum = 2 time units) with bounded-retry admission control on memory, and a fail-fast capacity check performed once at arrival. This was implemented as a fully working UNIX shell script (`enrolment_scheduler_`.sh), executed to produce a real, reproducible execution log, and analysed against nine enrolment requests including one personalised request (REQ_11, derived from the last two digits of Student ID).

The simulation demonstrates all required outcomes: successful execution, waiting under resource contention, and rejection for two distinct reasons — capacity exhaustion (REQ_03, REQ_08) and memory exhaustion (REQ_05, and the personalised REQ_11). The personalised request's failure is analysed in depth as a case study in resource starvation, and used to justify a concrete design recommendation (reserved memory partitioning for large requests) that a real VU systems team could apply.

1. The VU Enrolment System, In Our Own Words

At the start of each semester, a large number of students access the VU enrolment system within a short period of time. Students use the system to enrol in subjects, change tutorial classes, check seat availability and finalise their timetables. Although these actions seem simple from the student's side, every request triggers several background tasks such as checking prerequisites, detecting timetable clashes, confirming class capacity and updating student records.

When thousands of students submit requests at the same time, those requests compete for limited resources such as CPU processing time and memory. Since the system can only actively process one enrolment action at a time, it must constantly decide which request should be handled next, which requests need to wait, and which requests cannot be completed because system constraints have been reached.

For this project, we model the enrolment system as an operating system environment where each enrolment request behaves like a process. These processes compete for shared resources, and the scheduler is responsible for managing them fairly and efficiently while ensuring the system remains responsive and reliable during periods of heavy demand.

This is precisely the situation an operating system scheduler faces: many competing units of work (processes), a scarce processing resource (the CPU), a scarce memory resource, and an external constraint that no amount of processing power can overcome (in the OS world, a locked resource; here, a full class). Our project treats the enrolment platform exactly this way, and builds, runs, and analyses a real scheduler that makes these decisions transparently and consistently.

2. Enrolment Actions as Operating-System-Managed Activities

Each enrolment request in our simulation carries the same information an OS process control block would track: an arrival time, a CPU burst (the work required to validate it), a memory footprint (the space needed to hold the student's record while it is being processed), and a capacity flag representing whether a seat exists. The scheduler we built manages these nine requests exactly as an OS kernel manages processes — admitting them

into memory when space allows, giving each a fair, bounded slice of CPU time via Round Robin, and releasing memory the instant a request is fully processed.

Where our system goes beyond a single-resource model is that it must reason about CPU, memory, and capacity together, not independently. A request can pass its capacity check and still fail on memory (as REQ_05 and REQ_11 do in our simulation); a request can have plenty of memory space but still have to wait its turn on the CPU. Section 3 below unpacks exactly how each constraint shapes the outcome.

2a. Operating System Evolution and Social Impact of Scheduling Design

The problem the VU enrolment system faces is not new. It is the same core problem operating systems have faced since batch systems of the 1950s-60s, where jobs queued and ran strictly in submission order with no fairness guarantees, so a single long job could block every job behind it - exactly the failure mode FCFS would produce here, with REQ_02's six-unit burst blocking every shorter request behind it. Time-sharing systems of the 1960s-70s (e.g. CTSS, Multics, early UNIX) introduced preemptive, quantum-based CPU scheduling specifically to solve this problem - the direct ancestor of the Round Robin strategy used in this project. Modern networked and mobile operating systems extend the same idea further: a phone OS or cloud scheduler now balances CPU and memory alongside external constraints such as battery, network quota, and service limits simultaneously, which is why our scheduler cannot reason about CPU alone and must also enforce memory admission control and a hard capacity gate.

This history matters because scheduling design is not a purely technical choice - it carries a real social impact. A university enrolment system that used naive FCFS would give a structural advantage to whichever students' requests happened to be processed first, regardless of merit or need, effectively letting system architecture decide fairness of access to education. Round Robin combined with bounded admission retries is our attempt to make that fairness explicit and defensible rather than an accidental side effect of implementation order - the same reasoning that pushes real-world operating system designers toward scheduling policies that protect access, reliability, and scalability for all users, not just the fastest-arriving ones.

3. System Constraints and Their Effect on Enrolment Outcomes

3.1 CPU Processing Time

Only one enrolment action executes at any instant, per the assignment's system assumption. We schedule the CPU using Round Robin with a fixed quantum of 2 time units. This means no single computationally expensive action — such as a full timetable validation with a 6-unit burst — can occupy the CPU for its entire duration uninterrupted; it is preempted after 2 units and returned to the back of the ready queue, giving every other ready request a turn. This is what allows short actions like a 1-unit seat check (REQ_06) to complete promptly even while much larger requests are still in the system.

3.2 Memory Availability

Unlike the CPU, memory is not released the instant a request stops running on the processor — a request keeps holding its allocated memory for as long as it remains in the system (i.e. until it fully completes), because that memory represents its in-progress student record, not just its current CPU turn. This means several requests can be memory-resident at once even though only one of them is executing at any instant — a direct simulation of real multiprogramming, where memory (not CPU) is usually the binding constraint on how many processes the system can hold concurrently.

3.3 Class Capacity Limits

Class capacity is checked as soon as an enrolment request enters the system, before any memory or CPU resources are allocated to it. This is important because a full class cannot be fixed by waiting

longer or by making more memory available. If there are no seats left, the request cannot succeed regardless of how many system resources are available.

By checking capacity first, the system avoids wasting CPU time and memory on requests that are guaranteed to fail. This helps the scheduler focus its resources on requests that still have a chance of being processed successfully. In our simulation, REQ_03 and REQ_08 are both rejected immediately because their classes have no available seats, making further processing unnecessary.

4. Trade-offs

4.1 Fairness vs Correctness

Strict fairness would suggest giving every request an equal chance at memory, first-come-first-served. But our simulation shows this is not enough to guarantee correctness: REQ_11 arrives and is capacity-eligible, yet correctly admitting it would require denying memory to smaller, already-resident requests that could otherwise complete quickly. We resolved this by bounding each request to a fixed number of admission retries (MAX_WAIT_ROUNDS = 3) rather than letting any single request wait indefinitely or forcibly evicting resident requests — this keeps the system correct (never over-allocates memory) while remaining predictable and bounded (never leaves a request in limbo forever).

4.2 Responsiveness vs Validation Accuracy

A shorter Round Robin quantum improves responsiveness (short requests finish sooner) but increases the number of context switches the scheduler must log and reason about; a longer quantum reduces overhead but risks short requests waiting behind long ones for longer stretches. We selected a quantum of 2 time units as a balance point given our workload's CPU bursts (ranging from 1 to 6 units), verified empirically by inspecting the execution log: even the largest burst (REQ_02, 6 units) completes within 3 quanta, without ever blocking a smaller request for more than one quantum.

Section A — Multi-Constraint Reasoning

The table below classifies every enrolment request in our simulated workload against the three constraints (CPU, memory, capacity), based on the real, reproducible output in execution_log_.txt.

Request	Arrival	CPU	Mem	Seat	Outcome	Justification
REQ_01	0	4	20	Yes	Executed → Completed	Capacity OK; memory free immediately
REQ_02	0	6	30	Yes	Executed → Completed	Capacity OK; memory free immediately
REQ_03	1	2	15	No	Rejected	Failed capacity gate at arrival
REQ_04	1	3	25	Yes	Executed → Completed	Capacity OK; 25 ≤ 50 free at t=2
REQ_05	2	5	40	Yes	Waited → Rejected	Never saw ≥40 free within 3 retries
REQ_06	2	1	10	Yes	Executed → Completed	Small footprint fits alongside others
REQ_07	3	4	35	Yes	Executed → Completed	Admitted after REQ_01

Request	Arrival	CPU	Mem	Seat	Outcome	Justification
						freed memory
REQ_08	3	2	20	No	Rejected	Failed capacity gate at arrival
REQ_11*	4	3	90	Yes	Waited → Rejected	90-unit demand never satisfiable vs 100-unit pool

* REQ_11 is the personalised request (last two digits of Student ID).

Section B — Strategy Selection Under Conflict

We selected Round Robin for CPU scheduling combined with bounded-retry admission control for memory and a fail-fast gate for capacity. Round Robin was chosen over FCFS because FCFS would let a single long burst (e.g. REQ_02's 6 units) block every request behind it for its full duration. It was chosen over Shortest-Job-First because SJF requires advance knowledge of burst time and can starve long, legitimate requests indefinitely under continuous short-job arrivals. Priority scheduling was considered but not adopted at this stage, since the assignment brief does not define a legitimate priority ordering between enrolment types, and introducing one without justification risks arbitrary-looking outcomes. Round Robin, by contrast, guarantees bounded CPU wait time for every request without needing that information, satisfying both fairness and responsiveness simultaneously.

Section C — Simulation with Partial Failure

Using a time slice of 2 units and a limited total memory pool of 100 units, the table below traces the system state at each CPU dispatch event, drawn directly from the real script output.

Time	Event	Remaining CPU (req)	Memory free / 100
0	REQ_01, REQ_02 admitted	—	50
0-2	REQ_01 runs	REQ_01: 2	50
2	REQ_04 admitted; REQ_05 waits (retry 1)	—	25
2-4	REQ_02 runs	REQ_02: 4	25
4	REQ_05 waits (2); REQ_07 waits (1); REQ_11 waits (1)	—	15
4-6	REQ_01 runs (final quantum)	REQ_01: 0 → DONE	35
6	REQ_07 admitted; REQ_05 waits (3); REQ_11 waits (2)	—	0
6-8	REQ_04 runs	REQ_04: 1	0
8	REQ_05 REJECTED (memory, 3 retries exceeded)	—	0
8-9	REQ_06 runs (final quantum)	REQ_06: 0 → DONE	10
9	REQ_11 REJECTED (memory, 3 retries exceeded)	—	10

Time	Event	Remaining CPU (req)	Memory free / 100
9-18	REQ_02, REQ_07, REQ_04 complete in rotation	all → 0 → DONE	100 (all released)

At least one failed request is clearly demonstrated: REQ_05 fails at t=8 and the personalised REQ_11 fails at t=9, both for the same underlying reason — insufficient free memory after the maximum permitted admission retries — while two further requests (REQ_03, REQ_08) fail earlier, at arrival, for the unrelated reason of class capacity.

Section D — Reflection

Fairness does not mean treating every enrolment request exactly the same. Different requests place different demands on the system. For example, a simple seat availability check only requires a small amount of CPU time and memory, while a full timetable validation requires much more processing and memory resources. If every request was given unlimited access to system resources, a small number of large requests could use most of the available memory and prevent other requests from being processed. In that situation, the overall outcome would actually be less fair because more students would experience delays.

Our simulation demonstrates this clearly. REQ_11 was not rejected because it arrived late or because it was less important than other requests. It was rejected because it required 90 units of memory, and admitting it would have prevented several smaller requests from being processed successfully. As a result, the scheduler prioritised the requests that could be completed with the resources currently available.

Real operating systems make similar decisions every day when system resources become limited. Techniques such as priority levels, memory limits, bounded retry counts and admission control are used to ensure that one large process cannot prevent all other processes from making progress. The same principle is applied in our enrolment system through the MAX_WAIT_ROUNDS = 3 rule, which gives each request multiple opportunities to obtain memory before a final decision is made. This approach balances fairness, responsiveness and overall system stability.

Team Coordination

Throughout the project, we regularly discussed our progress, divided responsibilities and reviewed each other's work. Before implementation began, we agreed on the scheduling approach and system design. We also cross-checked the final report, script and evidence files to ensure they were consistent with the project requirements before submission.

Individual Contribution

Student	Contribution
Jenish Gurung ()	System design and shell script implementation (enrolment_scheduler_.sh); execution log, process snapshot, and failure case evidence generation; Sections 1-3 and Section A/C of the report.
Hoang Gia Phat Nguyen ()	Scheduling strategy comparison and justification (Section B); trade-off analysis (Section 4); reflection (Section D) and final report

Student	Contribution
	formatting/review.

Personalised Enrolment Request That Cannot Be Fulfilled

Request REQ_11, personalised using the last two digits of Student ID ("11"), arrives at t=4 requesting 90 of the system's 100 memory units. It passes the class-capacity check immediately (a seat is available) and has a modest CPU burst of only 3 units — on both of those dimensions it is entirely schedulable. It fails purely because of memory contention: by the time it arrives, four smaller requests (REQ_01, REQ_02, REQ_04, REQ_06) are already memory-resident, and the small amounts of memory freed as they complete (20 units, then 10 units) never come close to the 90 units REQ_11 needs, within the three admission retries the scheduler allows. It is formally rejected at t=9 with the logged reason "insufficient memory after 3 admission retries (needs 90, only 10/100 free)", visible directly in execution_log_.txt and analysed in detail in failure_case_explanation_.txt.